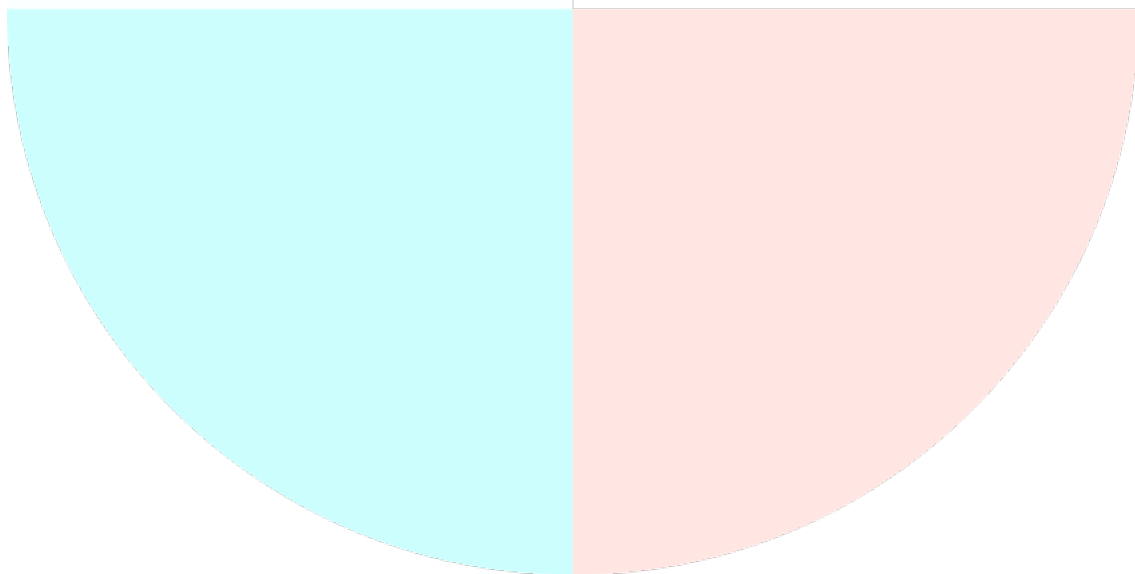


Bloque 2 - Tema 3 ejercicios

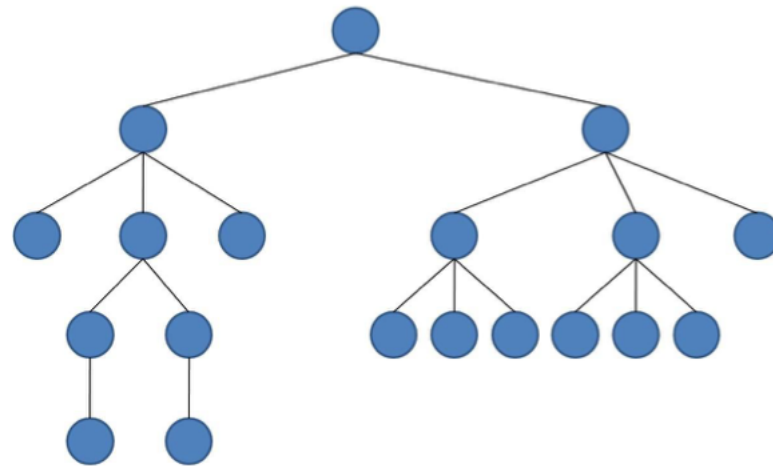
TIPOS ABSTRACTOS Y ESTRUCTURAS DE DATOS.
ORGANIZACIONES DE FICHEROS. ALGORITMOS.
FORMATOS DE INFORMACIÓN Y FICHEROS



PREPARACIÓN OPOSICIONES
TÉCNICOS AUXILIARES DE INFORMÁTICA

Enunciado 1

Explica los valores de las principales características del siguiente árbol:

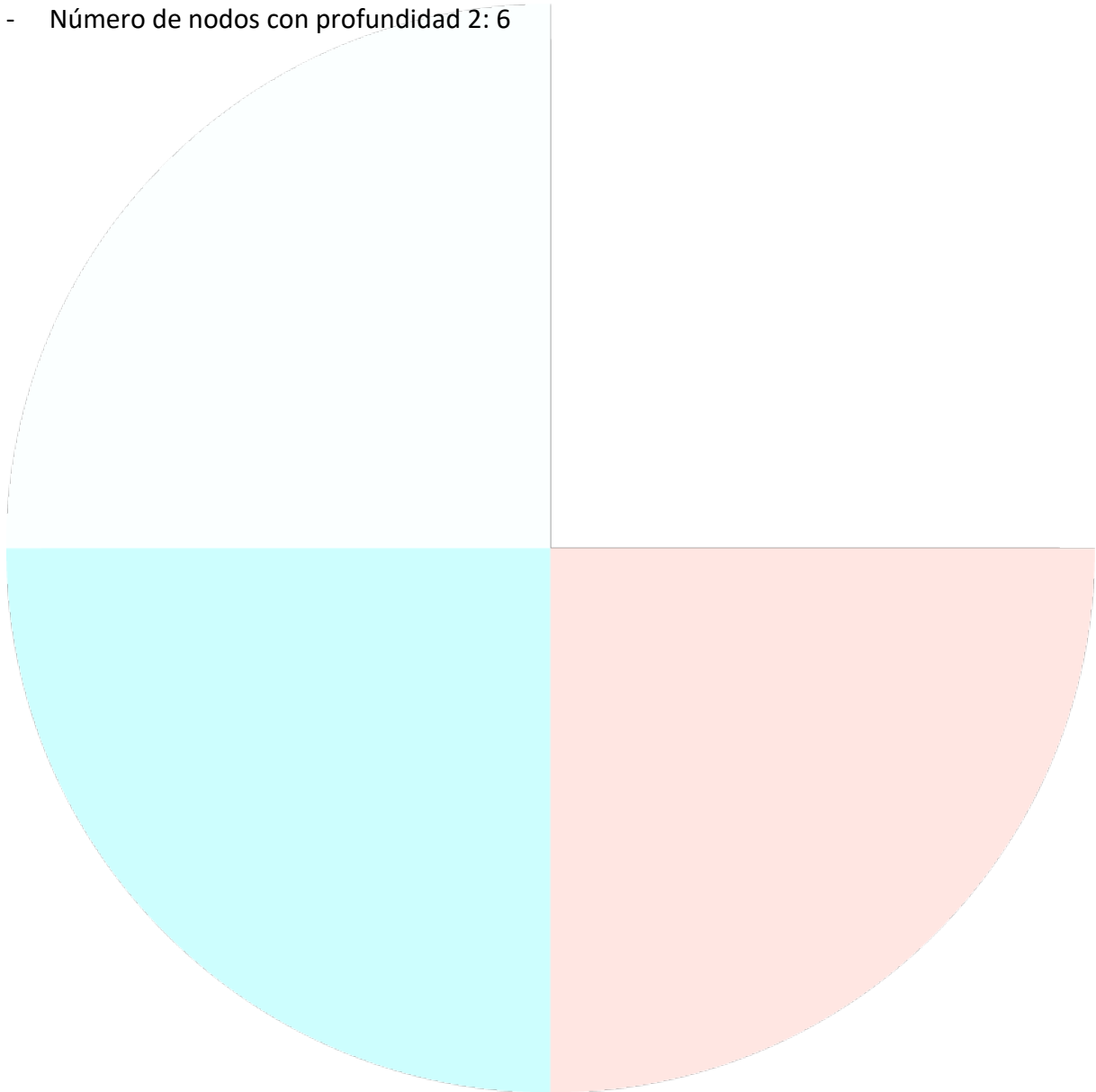


- Grado del árbol:
- Altura del árbol:
- Número nodos del árbol:
- Número de hojas:
- Número de nodos internos:

Enunciado 2

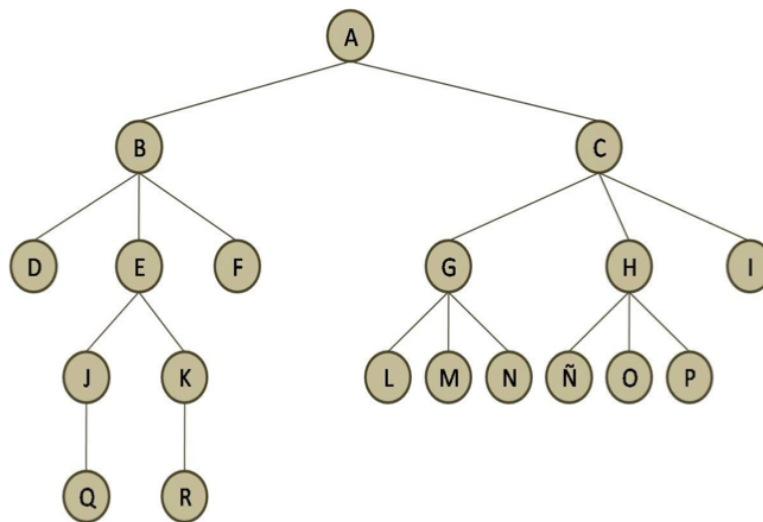
Dadas las siguientes propiedades de un árbol, proporcione un dibujo que satisfaga las mismas:

- Grado del árbol: 3
- Número de nodos: 14
- Altura del árbol: 4
- Número de nodos con profundidad 2: 6

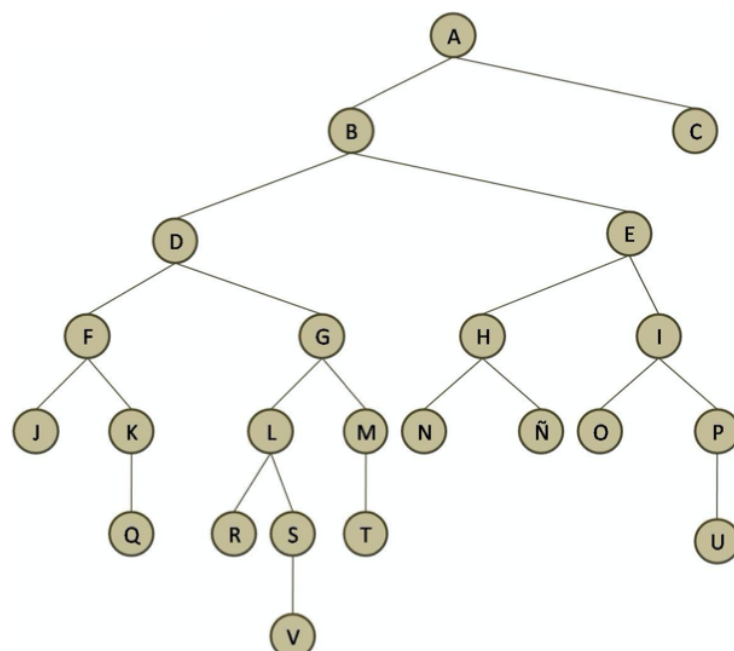


Enunciado 3

Dados los siguientes árboles, escriba los recorridos preorden, inorden, postorden y por niveles.



- Preorden:
- Inorden:
- Postorden:
- Por niveles:

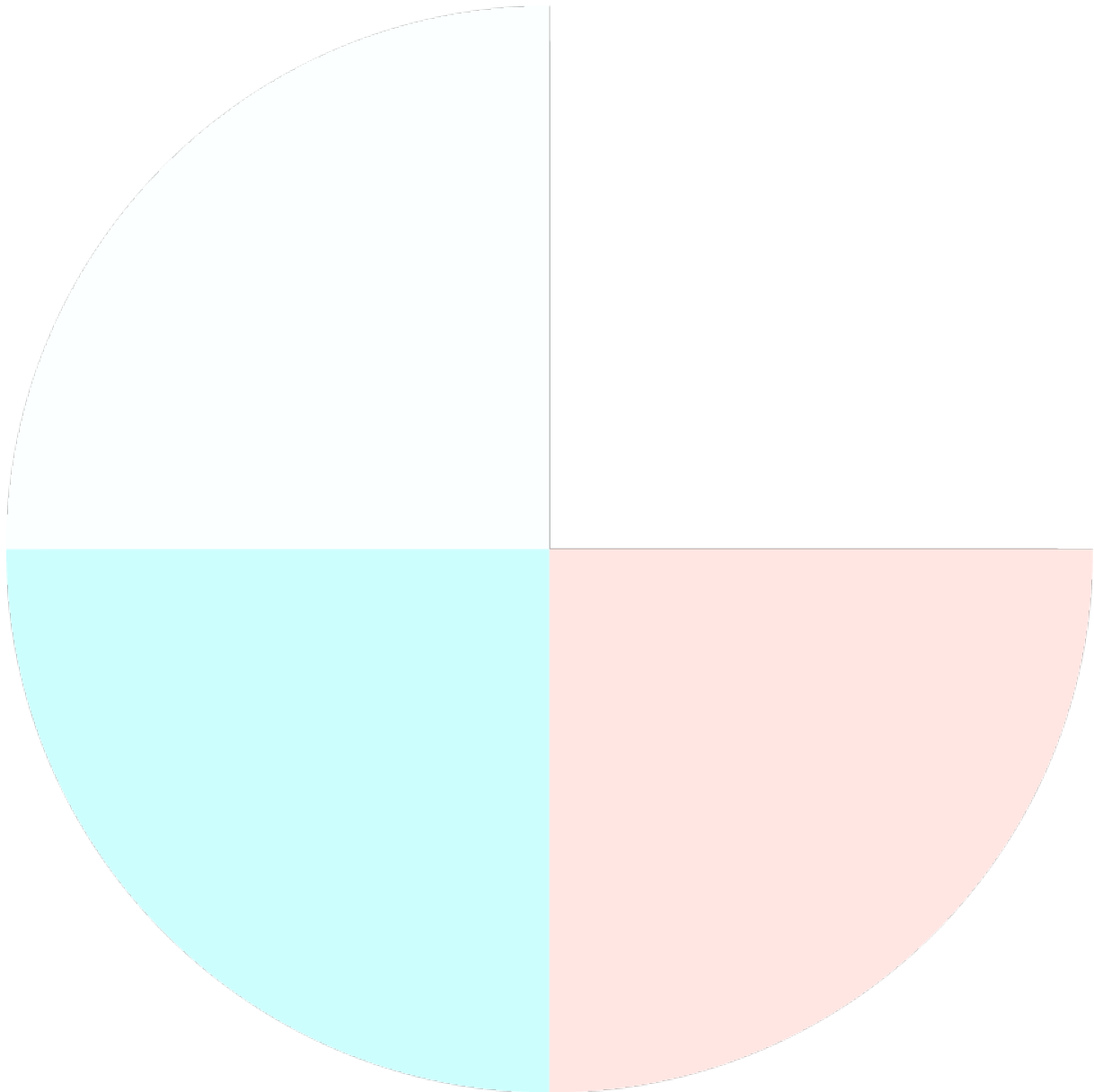


- Preorden:
- Inorden:
- Postorden:
- Por niveles:

Enunciado 4

El recorrido en preorden de un árbol binario es EXAMFUN y en inorden MAFXUEN. Se pide:

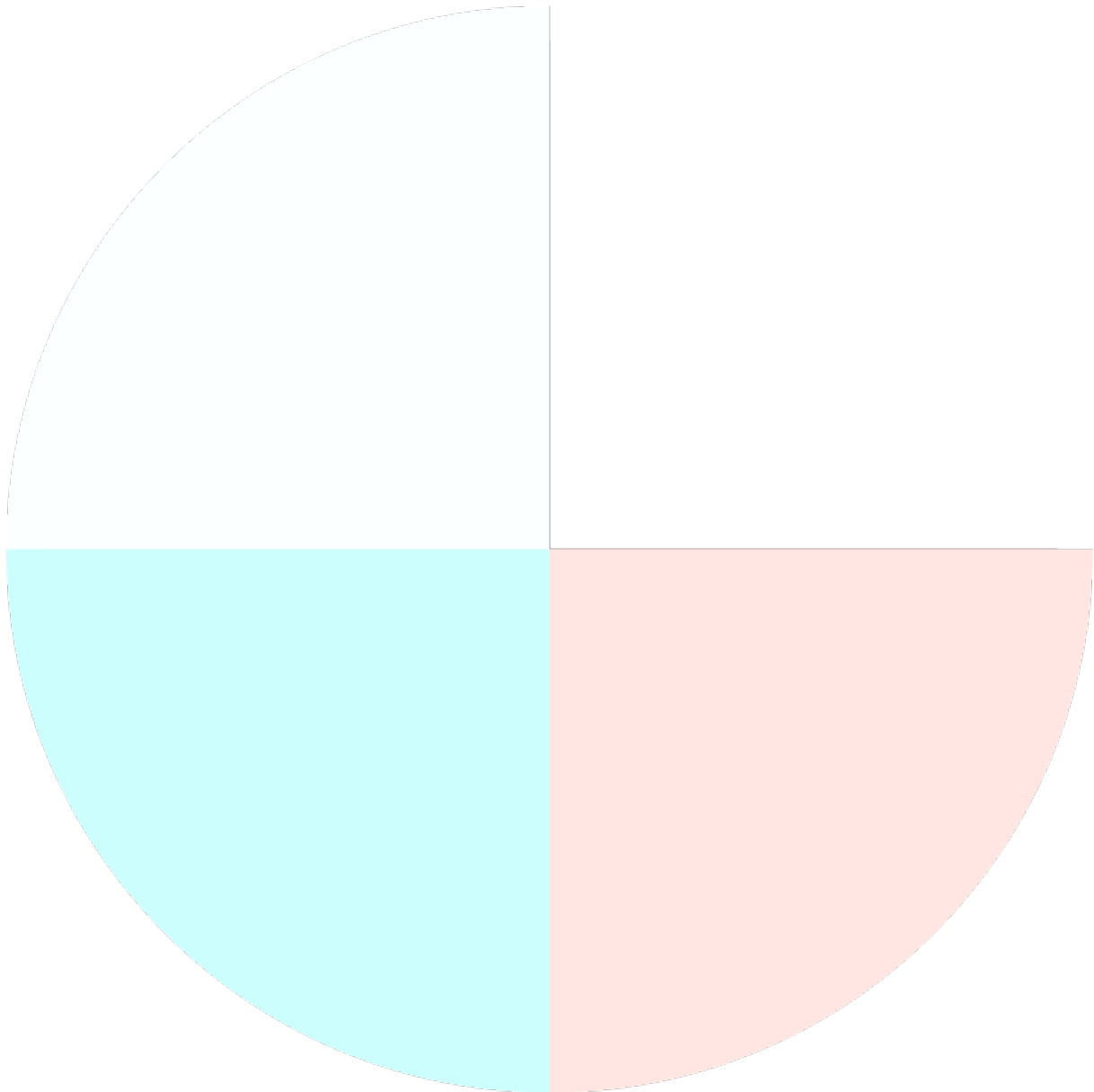
- Dibujar el árbol binario.
- Recorrido en postorden.
- Recorrido por niveles.



Enunciado 5

El recorrido en preorden de un árbol binario es GEAIBMCLDFKJH y en inorden IABEGLDCFMKHJ. Se pide:

- Dibujar el árbol binario.
- Recorrido en postorden.



Enunciado 6

- a) Escribir una función recursiva que devuelva el número de nodos de un árbol binario.
- b) Escribir una función recursiva que devuelva la altura del árbol binario.
- c) Construir una función que sume los elementos de un array de números enteros recursivamente.
- d) Escribir una función recursiva para la siguiente fórmula:

$$F(n) = \begin{cases} 1, & \text{si } n = 1; \\ 1, & \text{si } n = 2; \\ F(n-1) + F(n-2) & \text{si } n > 2; \end{cases}$$

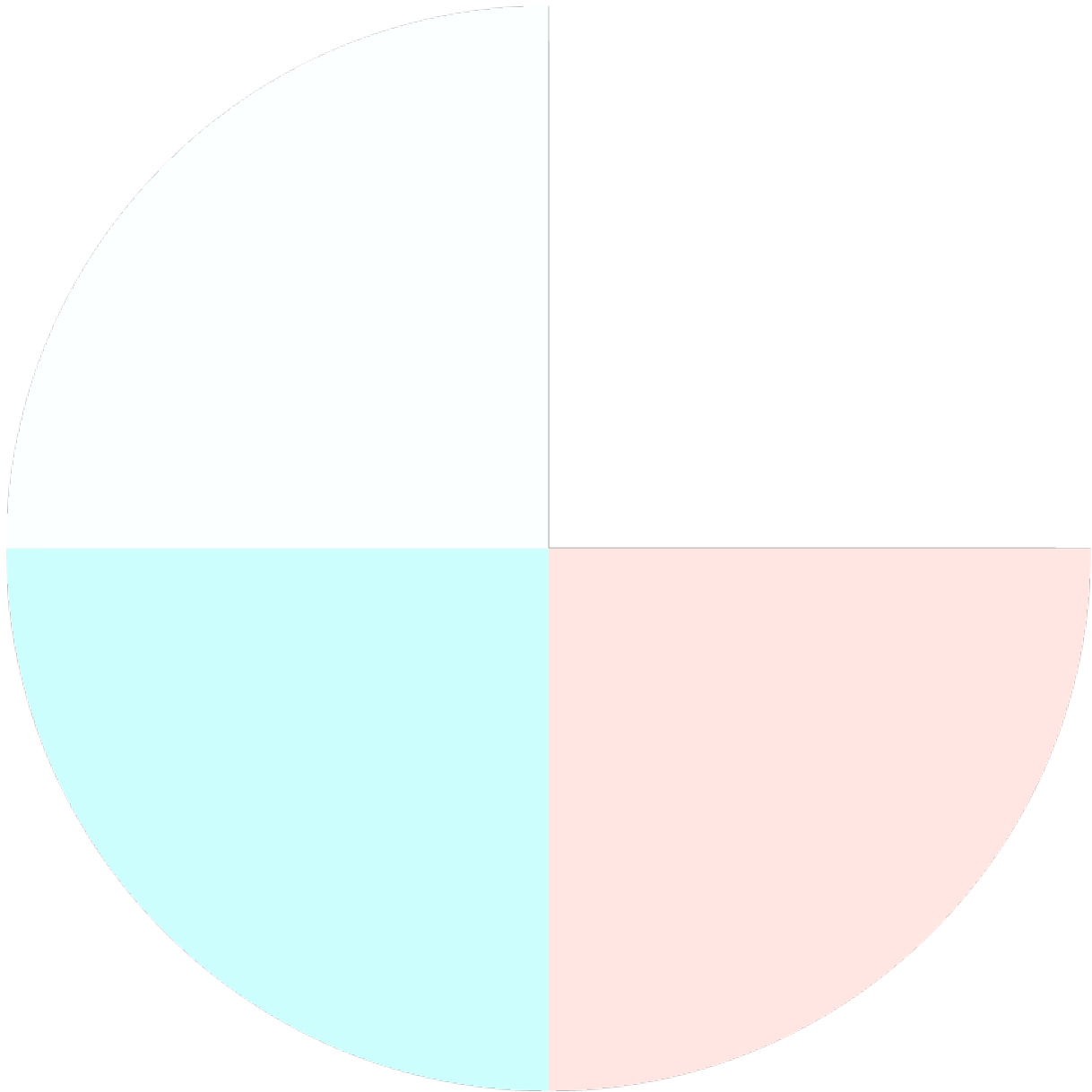
- e) Explique la ejecución del siguiente algoritmo para par(3), par(6) y par(7):

```
int par(int n){
    if (n==0) return 1;
    else return (impar(n-1));
}

int impar(int n){
    if (n==0) return 0;
    else return(par(n-1));
}
```

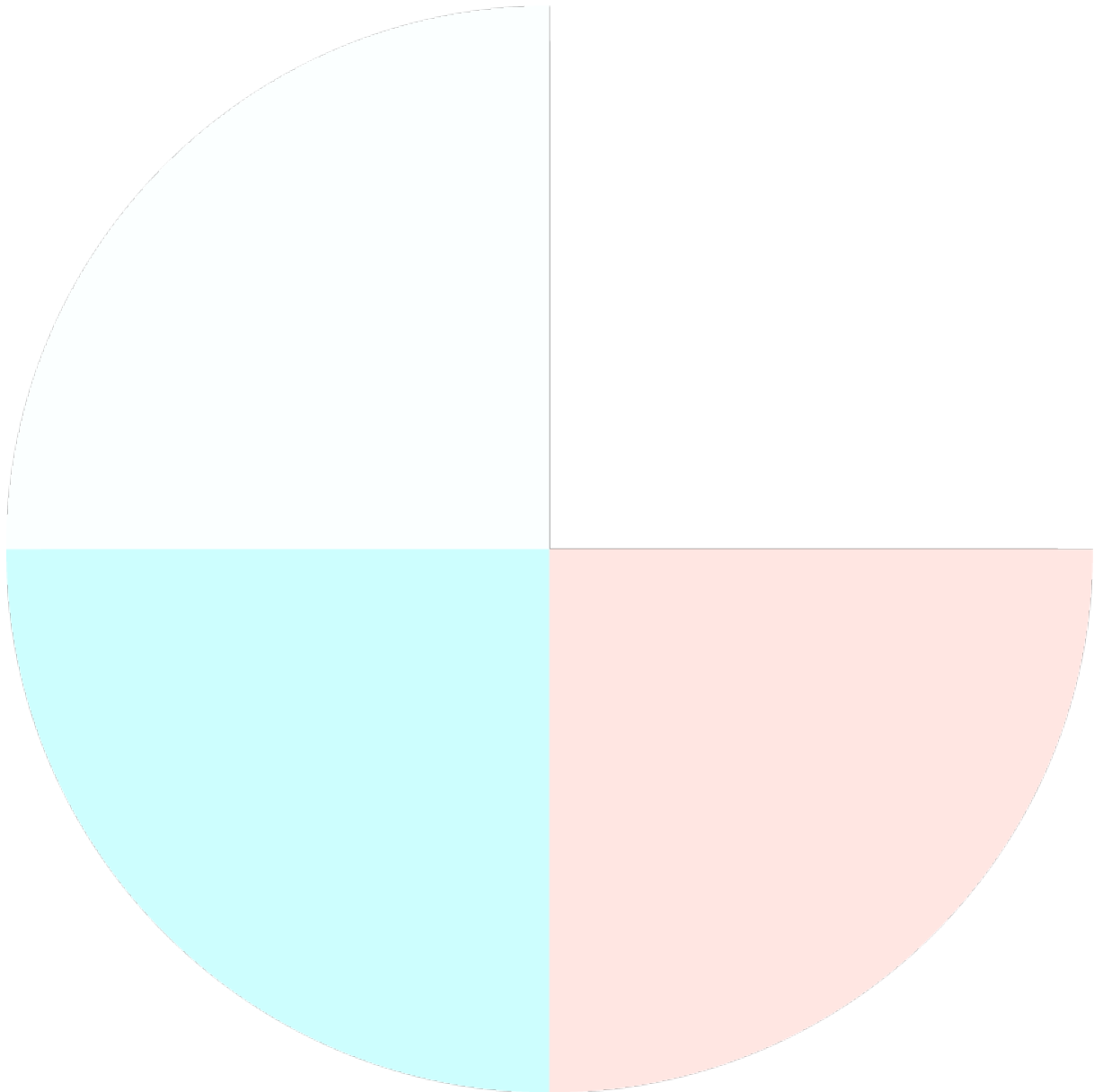
Enunciado 7

Construir un ABB con las claves 50, 25, 75, 10, 40, 60, 90, 35, 45, 70, 42.



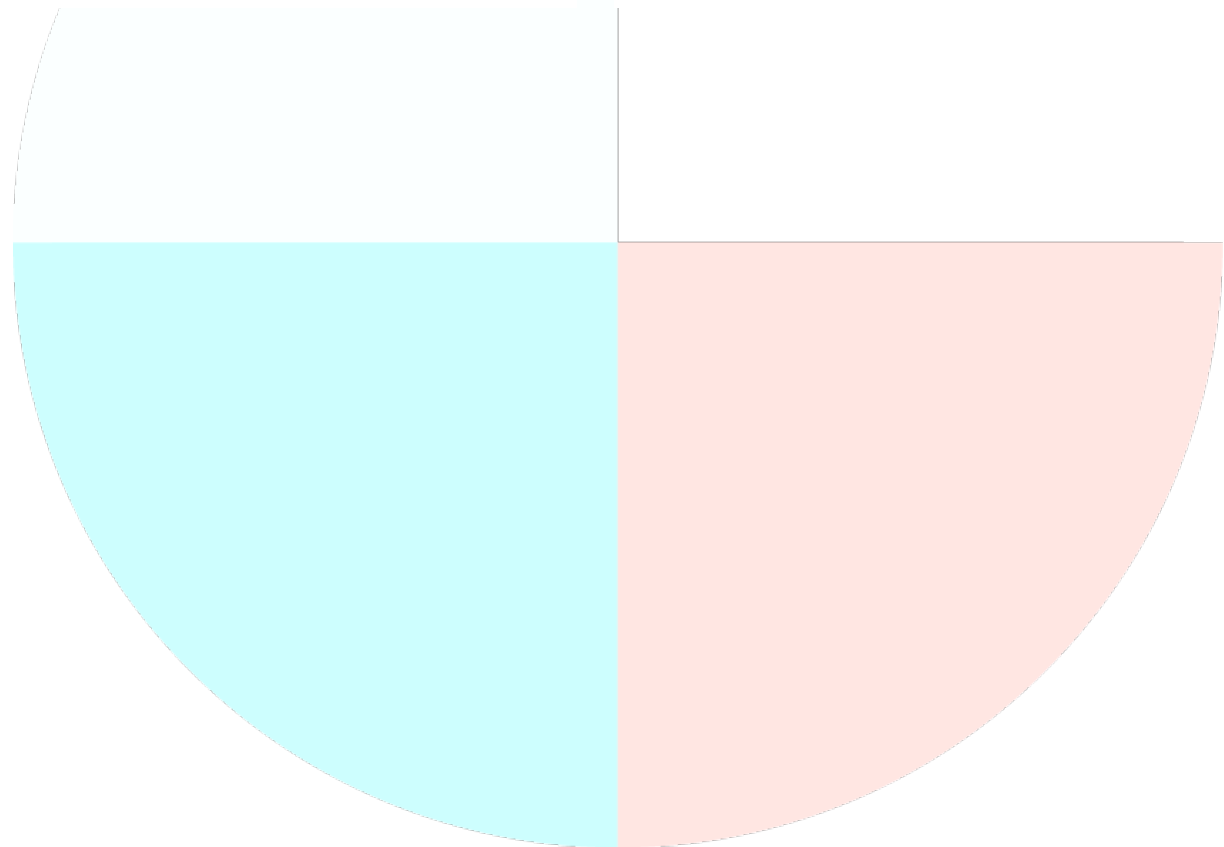
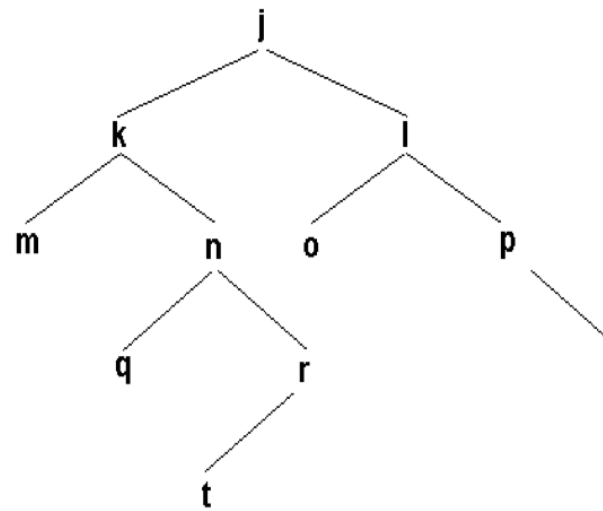
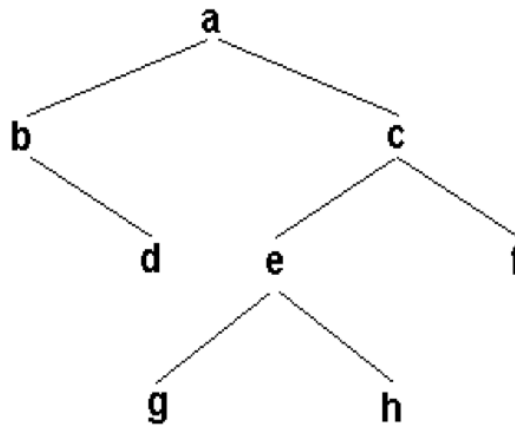
Enunciado 8

Construir un ABB equilibrado a partir de las claves 10, 75, 34, 22, 64, 53, 41, 5, 25, 74, 20, 15, 90.



Enunciado 9

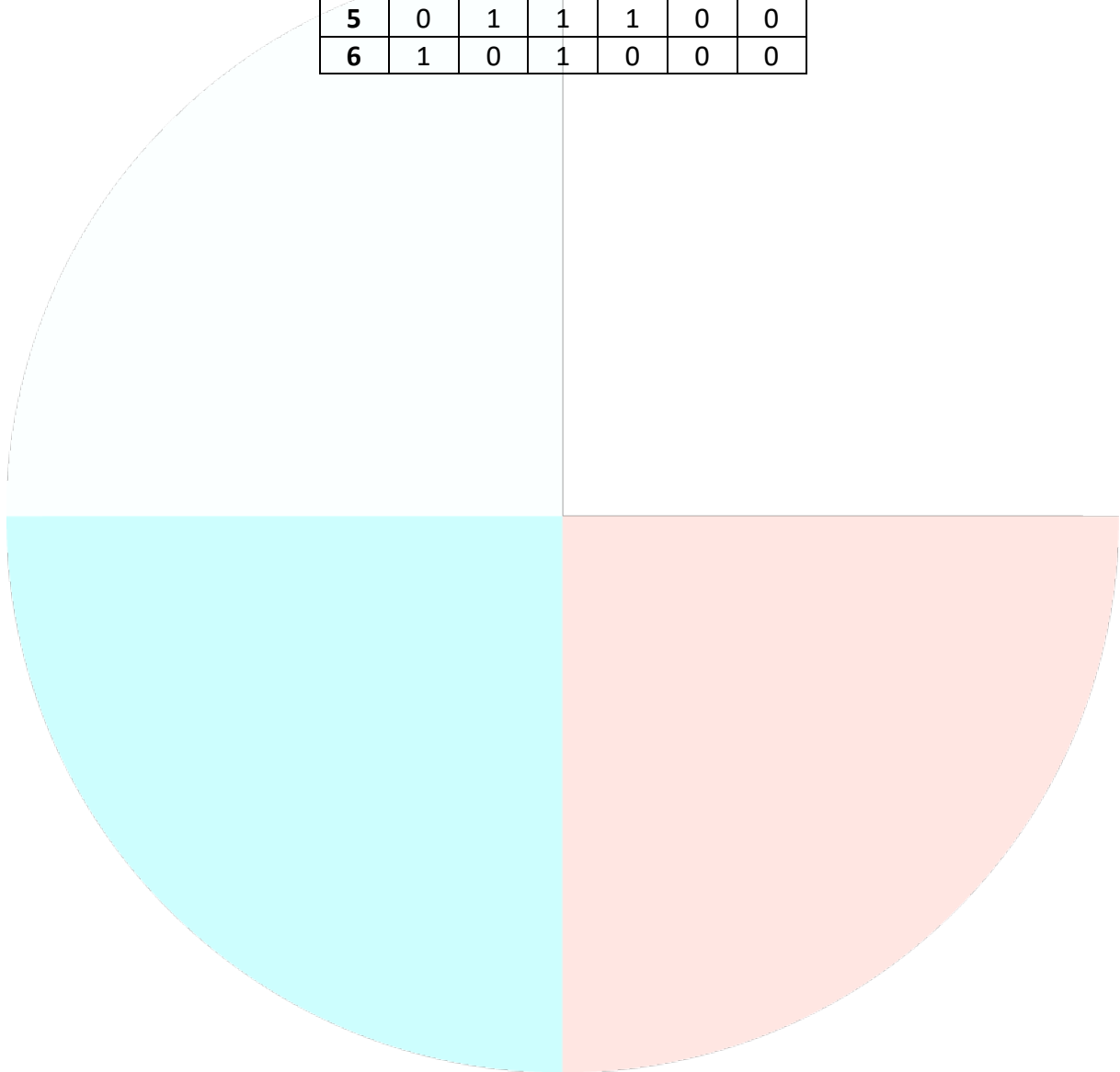
Indique cuál de los siguientes es un árbol AVL y justifique su respuesta:



Enunciado 10

A partir de la siguiente matriz de adyacencia construya el grafo asociado:

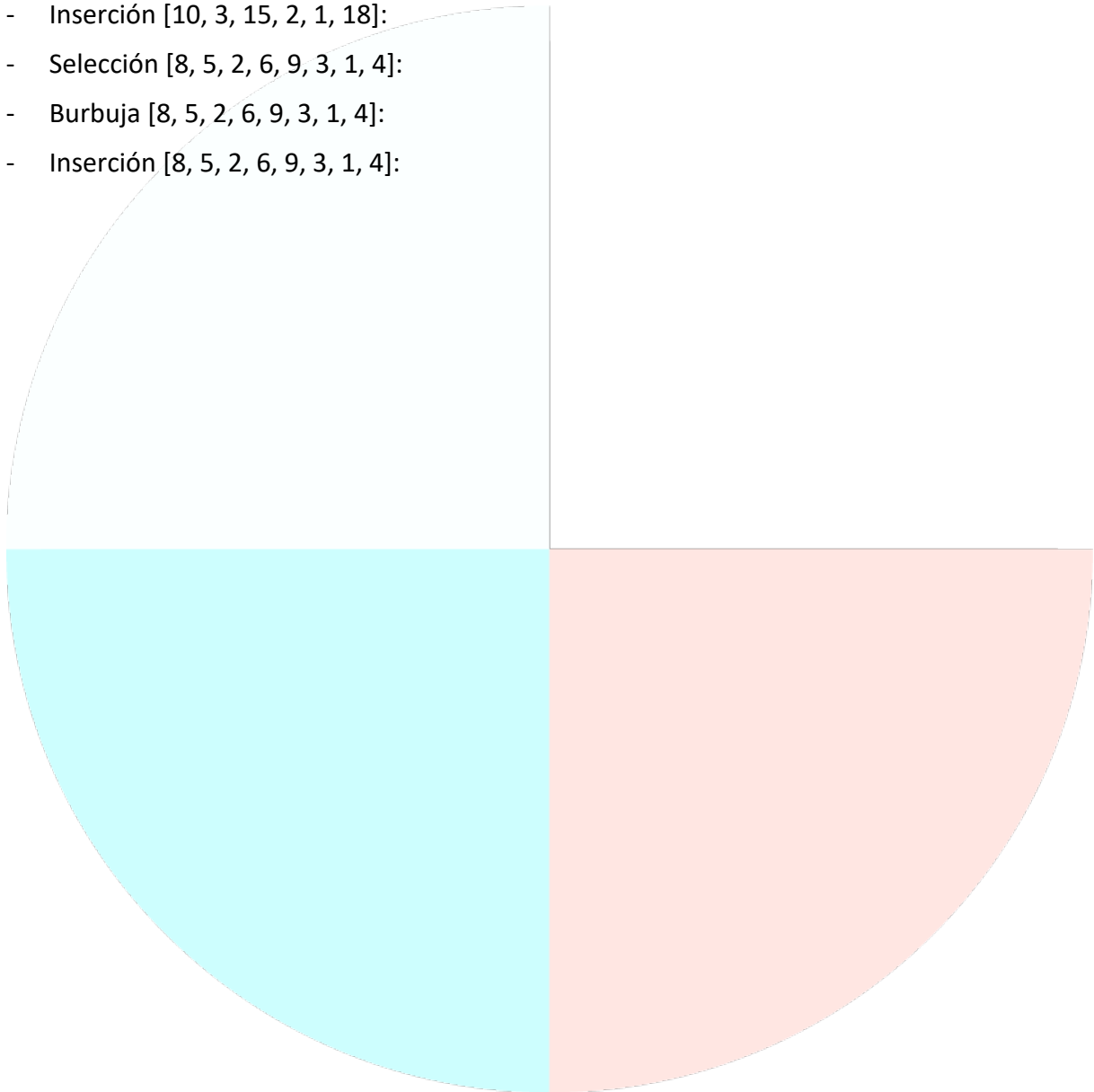
	1	2	3	4	5	6
1	0	0	1	1	0	1
2	0	0	1	1	1	0
3	1	1	0	0	1	1
4	1	1	0	0	1	0
5	0	1	1	1	0	0
6	1	0	1	0	0	0



Enunciado 11

¿Cuántas iteraciones deberá ejecutarse del algoritmo de ordenación para que quede totalmente ordenado en sentido ascendente?

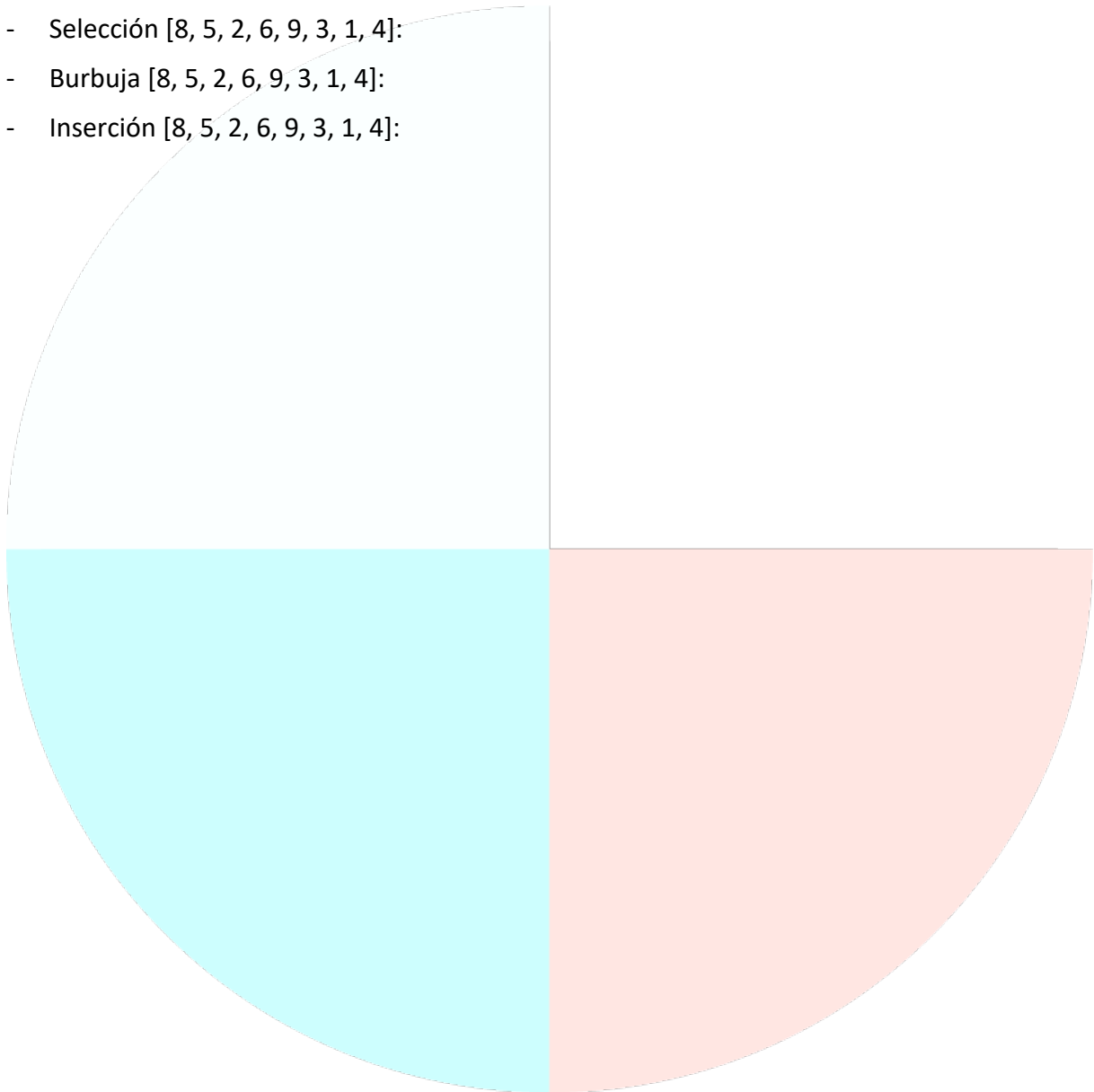
- Selección [10, 3, 15, 2, 1, 18]:
- Burbuja [10, 3, 15, 2, 1, 18]:
- Inserción [10, 3, 15, 2, 1, 18]:
- Selección [8, 5, 2, 6, 9, 3, 1, 4]:
- Burbuja [8, 5, 2, 6, 9, 3, 1, 4]:
- Inserción [8, 5, 2, 6, 9, 3, 1, 4]:



Enunciado 12

Represente el array en la TERCERA iteración del algoritmo de ordenación:

- Selección [10, 3, 15, 2, 1, 18]:
- Burbuja [10, 3, 15, 2, 1, 18]:
- Inserción [10, 3, 15, 2, 1, 18]:
- Selección [8, 5, 2, 6, 9, 3, 1, 4]:
- Burbuja [8, 5, 2, 6, 9, 3, 1, 4]:
- Inserción [8, 5, 2, 6, 9, 3, 1, 4]:



Enunciado 13

Usando la notación $O(\cdot)$, dar el peor caso de tiempo de ejecución de las siguientes funciones, expresándolo en función de n :

```
1: void ejemplo1(int n)
2: {
3:     int i, j, k;
4:
5:     for (i = 0; i < n; i++)
6:         for (j = 0; j < n; j++)
7:             {
8:                 C[i][j] = 0;
9:                 for (k = 0; k < n; k++)
10:                    C[i][j] = += A[j][k] * B[k][j];
11:             }
12: }
```

```
1: void ejemplo2(int n)
2: {
3:     int i, j, k;
4:
5:     for (i = 1; i < n; i++)
6:         for (j = i+1; j <= n; j++)
7:             for (k = 1; k <= j; k++)
8:                 Global += k*i;
9: }
```

```
1: void ejemplo3(int n)
2: {
3:     int i, j, x, y;
4:
5:     for (i = 1; i <= n; i++)
6:         for (i % 2 == 1)
7:             {
8:                 for (j = i; j <= n; j++)
9:                     x++;
10:                 for (j = 0; j < i; j++)
11:                     y++;
12:             }
13: }
```

Enunciado 14

Usando la notación $O(\cdot)$, obtener el peor caso de tiempo de ejecución para las siguientes funciones, expresándolo en función de n :

```
1: int recursiva(int n)
2: {
3:     if (n <= 1)
4:         return 1;
5:     else
6:         return (recursiva(n - 1) + recursiva(n - 1));
7: }
```

```
1: int E(int n)
2: {
3:     if (n == 1)
4:         return n;
5:     else
6:         return (E(n/2) + 1);
7: }
```

